

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5

9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

*I Lokesh S (192511037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Aim

To design a complete pseudocode algorithm that accepts an SQL query, parses and validates it, optimizes the query, generates an execution plan, executes it, and produces the final result.

Pseudocode

ALGORITHM COMPLETE_QUERY_PROCESSING

INPUT:

SQL_Query

OUTPUT:

Query_Result

BEGIN

READ SQL_Query

IF SQL_Query = EMPTY THEN

PRINT "Error: Query cannot be empty"

STOP

END IF

// Step 1: Parsing

Tokens ← TOKENIZE(SQL_Query)

Parse_Tree ← PARSE(Tokens)

IF Parse_Tree = INVALID THEN

PRINT "Error: Invalid SQL syntax"

STOP

END IF

// Step 2: Validation

IF NOT VALIDATE_TABLES(Parse_Tree) THEN

```

    PRINT "Error: Invalid table"
    STOP
END IF

IF NOT VALIDATE_COLUMNS(Parse_Tree) THEN
    PRINT "Error: Invalid column"
    STOP
END IF

// Step 3: Convert to relational algebra
RA_Query ← CONVERT_TO_RELATIONAL_ALGEBRA(Parse_Tree)

// Step 4: Query optimization
RA_Query ← PUSH_SELECTION_DOWN(RA_Query)
RA_Query ← PUSH_PROJECTION_DOWN(RA_Query)
RA_Query ← OPTIMIZE_JOIN_ORDER(RA_Query)
RA_Query ← SELECT_BEST_JOIN_METHOD(RA_Query)

// Step 5: Generate execution plan
Execution_Plan ← GENERATE_EXECUTION_PLAN(RA_Query)

// Step 6: Execute query
Query_Result ← EXECUTE(Execution_Plan)

// Step 7: Display result
IF Query_Result = EMPTY THEN
    PRINT "No records found"
ELSE
    DISPLAY Query_Result
END IF

END

Query Optimization Procedure
PROCEDURE OPTIMIZE_QUERY(RA_Query)

```

BEGIN

RA_Query $\leftarrow$ PUSH_SELECTION_DOWN(RA_Query)

RA_Query $\leftarrow$ PUSH_PROJECTION_DOWN(RA_Query)

RA_Query $\leftarrow$ OPTIMIZE_JOIN_ORDER(RA_Query)

RA_Query $\leftarrow$ SELECT_BEST_JOIN_METHOD(RA_Query)

RETURN RA_Query

END

Execution Procedure

PROCEDURE EXECUTE(Execution_Plan)

BEGIN

FOR EACH Operation IN Execution_Plan DO

IF Operation = SELECTION THEN

 APPLY_SELECTION()

ELSE IF Operation = PROJECTION THEN

 APPLY_PROJECTION()

ELSE IF Operation = JOIN THEN

 APPLY_JOIN()

ELSE IF Operation = SORT THEN

 APPLY_SORT()

END IF

END FOR

RETURN Final_Result

END

Flow

SQL Input



Tokenization



Parsing



Validation



Relational Algebra



Query Optimization



Execution Plan



Query Execution



Result Output

Result

The algorithm successfully processes an SQL query from **input** → **parsing** → **validation** → **relational algebra** → **optimization** → **execution plan** → **execution** → **final result**. It also handles invalid or empty queries and applies selection, projection, join-order, and join-method optimizations, satisfying the requirements of a complete Level 4 algorithm.

Q2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Aim

To design a cost-based query optimization algorithm that estimates the **I/O cost of different join orderings** and selects the join order with the minimum estimated cost.

Pseudocode

ALGORITHM COST_BASED_QUERY_OPTIMIZATION

INPUT:

Relations R1, R2, ..., Rn

Number of pages for each relation

Join conditions

OUTPUT:

Best_Join_Order

BEGIN

Read all relations and their page counts

Best_Cost $\leftarrow$ INFINITY

Best_Join_Order $\leftarrow$ NULL

Generate all possible join orderings

FOR EACH Join_Order DO

Total_Cost $\leftarrow$ 0

Current_Relation $\leftarrow$ First relation in Join_Order

FOR EACH Next_Relation in Join_Order DO

Cost $\leftarrow$ ESTIMATE_JOIN_COST(
Current_Relation,

```

        Next_Relation
    )

    Total_Cost  $\leftarrow$  Total_Cost + Cost

    Current_Relation  $\leftarrow$  ESTIMATE_RESULT(
        Current_Relation,
        Next_Relation
    )

END FOR

IF Total_Cost < Best_Cost THEN
    Best_Cost  $\leftarrow$  Total_Cost
    Best_Join_Order  $\leftarrow$  Join_Order
END IF

END FOR

PRINT "Best Join Order:", Best_Join_Order
PRINT "Minimum I/O Cost:", Best_Cost

RETURN Best_Join_Order

END

```

Cost Estimation Procedure

```

FUNCTION ESTIMATE_JOIN_COST(R1, R2)

```

```

BEGIN

```

```

    B1  $\leftarrow$  Number of pages of R1

```

```

    B2  $\leftarrow$  Number of pages of R2

```


// Example: Hash Join cost

Cost $\leftarrow 3 \times (B1 + B2)$

RETURN Cost

END

Join Ordering Procedure

FUNCTION ESTIMATE_RESULT(R1, R2)

BEGIN

Estimate number of tuples

Estimate selectivity of join condition

Calculate size of intermediate result

RETURN Estimated_Result

END

Example

Suppose:

Employee = 100 pages

Department = 10 pages

Project = 50 pages

Consider two join orders:

Order 1: (Employee JOIN Department) JOIN Project

Order 2: (Department JOIN Project) JOIN Employee

The optimizer calculates the cost of both orders and compares their total I/O costs.

IF Cost(Order 1) < Cost(Order 2)

Select Order 1

ELSE

Select Order 2

END IF

The optimizer therefore chooses the **join ordering with the lowest estimated I/O cost**.

Result

The algorithm generates possible join orderings, estimates the I/O cost of each ordering, compares their costs, and selects the **least-cost join order** as the optimal query execution strategy. This reduces unnecessary I/O and improves query-processing performance.

Q3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Aim

To design a pseudocode algorithm for performing a **Nested Loop Join** between two relations and analyze its cost using the number of buffer pages and tuples.

Pseudocode

ALGORITHM NESTED_LOOP_JOIN(R, S)

INPUT:

Relation R

Relation S

Join condition $R.key = S.key$

OUTPUT:

Joined relation Result

BEGIN

Result $\leftarrow$ EMPTY

FOR EACH tuple r IN R DO

FOR EACH tuple s IN S DO

IF r.key = s.key THEN

 Add (r, s) to Result

END IF

END FOR

END FOR

RETURN Result

END

Block Nested Loop Join

For database systems, a **block/page-based nested loop join** is more efficient than comparing individual tuples one at a time.

ALGORITHM BLOCK_NESTED_LOOP_JOIN(R, S)

INPUT:

 Relations R and S

 B = Number of available buffer pages

OUTPUT:

 Joined relation Result

BEGIN

 Result $\leftarrow$ EMPTY

 FOR EACH block BR of R using (B - 2) buffers DO

 FOR EACH block BS of S DO

 FOR EACH tuple r IN BR DO

```

FOR EACH tuple s IN BS DO

    IF r.key = s.key THEN
        ADD (r, s) TO Result
    END IF

END FOR

END FOR

END FOR

END FOR

RETURN Result

END

```

Time / I/O Complexity

Let:

- b_R = number of pages in relation R
- b_S = number of pages in relation S
- B = number of available buffer pages

For Block Nested Loop Join:

$$\text{Cost} = b_R + \text{ceil}(b_R / (B - 2)) \times b_S$$

If R is the outer relation.

The approximate cost is:

$$O(b_R + (b_R / (B - 2)) \times b_S)$$

If tuple-by-tuple nested loops are considered, with:

- n_R = number of tuples in R
- n_S = number of tuples in S

then the comparison complexity is:

$$O(n_R \times n_S)$$

Example

Suppose:

R = 100 pages

S = 50 pages

B = 12 buffer pages

Then:

$$\begin{aligned}\text{Cost} &= 100 + \text{ceil}(100 / 10) \times 50 \\ &= 100 + 10 \times 50 \\ &= 600 \text{ I/O}\end{aligned}$$

Result

The Nested Loop Join compares tuples from the two relations and produces matching tuples. Its tuple comparison complexity is $O(nR \times nS)$, while the block-based I/O cost depends on the number of relation pages and available buffer pages. Increasing the number of buffers reduces the number of scans of the inner relation.

Q4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

The question is from **Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)**, Q4, worth 5 marks.

Pseudocode

ALGORITHM Detect_Deadlock(WFG)

INPUT:

Wait-For Graph G(V, E)

OUTPUT:

Deadlock detected or No deadlock

BEGIN

FOR each transaction T in V

```
    visited[T] ← FALSE
    recStack[T] ← FALSE
END FOR

FOR each transaction T in V
    IF visited[T] = FALSE THEN
        IF DFS(T) = TRUE THEN
            PRINT "Deadlock Detected"
            RETURN
        END IF
    END IF
END FOR
```

```
PRINT "No Deadlock"
```

```
END
```

```
FUNCTION DFS(T)
```

```
    visited[T] ← TRUE
    recStack[T] ← TRUE
```

```
    FOR each transaction U connected from T
```

```
        IF visited[U] = FALSE THEN
            IF DFS(U) = TRUE THEN
                RETURN TRUE
            END IF
        END IF
```

```
    ELSE IF recStack[U] = TRUE THEN
        RETURN TRUE
    END IF
```

END FOR

recStack[T] ← FALSE

RETURN FALSE

END FUNCTION

Example

Consider the Wait-For Graph:

T1 → T2

T2 → T3

T3 → T1

This forms a cycle:

T1 → T2 → T3 → T1

Therefore:

Deadlock Detected

Result

The algorithm uses **DFS and cycle detection** to identify deadlocks. If a cycle exists in the Wait-For Graph, the transactions involved are in a deadlock; otherwise, the system has no deadlock.

Q5. Design a pseudocode for the Basic Two-Phase Locking (2PL) protocol including lock request, grant, wait, and release phases.

This question asks for the **Basic 2PL protocol**, specifically covering **lock request, lock grant, waiting, and lock release**.

Pseudocode

ALGORITHM Basic_2PL(Transaction T)

BEGIN

// Growing Phase

WHILE T needs a data item DO

 REQUEST_LOCK(T, DataItem)

 IF Lock is available THEN

 GRANT_LOCK(T, DataItem)

 Perform Read/Write operation

 ELSE

 WAIT until Lock becomes available

 END IF

END WHILE

// Shrinking Phase

FOR each Lock held by T DO

 RELEASE_LOCK(T, DataItem)

END FOR

COMMIT T

END

FUNCTION REQUEST_LOCK(T, DataItem)

 IF DataItem is not locked THEN

 GRANT_LOCK(T, DataItem)

 ELSE

 WAIT(T)

 END IF

END FUNCTION

FUNCTION RELEASE_LOCK(T, DataItem)

 Unlock DataItem

 Wake up waiting transactions

END FUNCTION

Example

Consider:

T1 needs A

T2 needs A

Execution:

T1 → Request Lock(A)

 → Lock granted

 → Read/Write A

T2 → Request Lock(A)

 → A is locked by T1

 → T2 waits

T1 → Release Lock(A)

T2 → Lock(A) granted

 → Read/Write A

 → Release Lock(A)

Phases of Basic 2PL

1. Growing Phase

- Transaction can request and acquire locks.
- It cannot release locks.

2. Shrinking Phase

- Transaction releases its locks.
- It cannot acquire any new locks after releasing the first lock.

-

Result

Basic 2PL controls concurrent transactions by ensuring that locks are **requested and granted during the growing phase** and **released during the shrinking phase**. This helps maintain **conflict serializability**, although basic 2PL can still lead to deadlocks.

Q6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

This question asks for a **timestamp-based concurrency control algorithm using Thomas' Write Rule** to handle conflicting operations.

Pseudocode

ALGORITHM Thomas_Write_Rule(T, X, Operation)

INPUT:

Transaction T

Data item X

Operation = READ or WRITE

Each transaction has:

$TS(T)$ = Transaction timestamp

Each data item X has:

$R_TS(X)$ = Largest timestamp of a transaction that read X

$W_TS(X)$ = Largest timestamp of a transaction that wrote X

BEGIN

IF Operation = READ THEN

IF $TS(T) < W_TS(X)$ THEN

PRINT "Read rejected"

ABORT T

ELSE

 Perform READ(X)

$R_TS(X) \leftarrow \text{MAX}(R_TS(X), TS(T))$

END IF

ELSE IF Operation = WRITE THEN

 IF $TS(T) < R_TS(X)$ THEN

 PRINT "Write rejected"

 ABORT T

 ELSE IF $TS(T) < W_TS(X)$ THEN

 PRINT "Write ignored using Thomas' Write Rule"

 ELSE

 Perform WRITE(X)

$W_TS(X) \leftarrow TS(T)$

 END IF

END IF

END

Example

Suppose:

$TS(T1) = 10$

$TS(T2) = 20$

If T2 has already written X:

$W_TS(X) = 20$

and later T1 tries:

WRITE(X)

Since:

$TS(T1) < W_TS(X)$

$10 < 20$

Thomas' Write Rule says the write from T1 is **obsolete**, so it can be **ignored instead of aborting T1**.

Main Rules

Operation	Condition	Action
READ(X)	$TS(T) < W_TS(X)$	Abort
READ(X)	Otherwise	Read and update R_TS(X)
WRITE(X)	$TS(T) < R_TS(X)$	Abort
WRITE(X)	$TS(T) < W_TS(X)$	Ignore write
WRITE(X)	Otherwise	Perform write

Result

Thomas' Write Rule improves timestamp ordering by **ignoring obsolete write operations** when $TS(T) < W_TS(X)$, instead of unnecessarily aborting the transaction. This allows more transactions to complete while maintaining consistency.

Q7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

This question asks for the **ARIES recovery technique** using its three main phases: **Analysis, Redo, and Undo**.

Pseudocode

ALGORITHM ARIES_Recovery(Log)

INPUT:

Transaction Log

OUTPUT:

Recovered and Consistent Database

BEGIN

// Phase 1: ANALYSIS

Read the log from the last checkpoint

Identify:

Active transactions

Committed transactions

Dirty pages

Determine the point from which REDO must begin

// Phase 2: REDO

FOR each log record from the REDO starting point DO

IF the operation needs to be reapplied THEN

REDO the operation

END IF

END FOR

// Phase 3: UNDO

FOR each uncommitted transaction DO

Read its log records in reverse order

FOR each operation DO

UNDO the operation

END FOR

Write an ABORT/END record

END FOR

Flush recovery information to disk

PRINT "Database Recovery Completed"

END

ARIES Recovery Phases

1. Analysis Phase

Read log



Find active transactions



Find dirty pages



Determine REDO starting point

2. Redo Phase

Start from REDO point



Repeat required logged operations



Restore database state

3. Undo Phase

Find uncommitted transactions



Process their log records backwards



Undo their operations



Write compensation/recovery information

Example

Suppose the log contains:

<T1 START>

<T1, A, 100, 150>

<T2 START>

<T2, B, 200, 250>

<T1 COMMIT>

SYSTEM CRASH

During recovery:

Analysis → T1 committed, T2 uncommitted

Redo → Redo T1's update

Undo → Undo T2's update

Result

ARIES recovers the database by first **analyzing the log**, then **redoing necessary operations**, and finally **undoing the effects of incomplete transactions**. This restores the database to a consistent state after a system failure.

Q8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

This question asks for a **Shadow Paging recovery technique** in which the system maintains **current and shadow page tables** and rolls back to the shadow table if a failure occurs.

Pseudocode

ALGORITHM Shadow_Paging_Recovery

INPUT:

Database pages

OUTPUT:

Recovered database

BEGIN

Create Shadow_Page_Table

Create Current_Page_Table

Copy Current_Page_Table to Shadow_Page_Table

WHILE transaction is running DO

IF a page needs to be modified THEN

Create a new copy of the page

Update the new page

Update Current_Page_Table
to point to the new page

END IF

END WHILE

IF transaction COMMIT occurs THEN

Make Current_Page_Table permanent

Delete old Shadow_Page_Table

ELSE IF SYSTEM FAILURE occurs THEN

Current_Page_Table $\leftarrow$ Shadow_Page_Table

Discard modified pages

PRINT "Database Rolled Back"

END IF

END

Working

Before Transaction

Shadow Page Table



Old Pages

Current Page Table



Old Pages

When a page is modified:

Shadow Table → Old Page

Current Table → New Modified Page

On Commit

Current Page Table



becomes permanent

The transaction's changes are retained.

On System Failure

System Failure



Use Shadow Page Table



Discard Current Page Table



Restore Old Pages

Result

Shadow Paging provides recovery without requiring traditional **UNDO/REDO logging**. If the transaction commits, the current page table becomes permanent. If a system failure occurs, the database **rolls back to the shadow page table**, restoring the previous consistent state.

Q9. Write pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

This question asks for an **Immediate Update recovery algorithm** that uses **UNDO and REDO operations** on the log after a system crash.

Pseudocode

ALGORITHM Immediate_Update_Recovery(Log)

INPUT:

Transaction Log

OUTPUT:

Recovered Database

BEGIN

Read the log from the beginning

Identify:

Committed transactions

Uncommitted transactions

```
// REDO Phase
FOR each committed transaction T DO

    FOR each log record of T
        REDO the operation
    END FOR

END FOR

// UNDO Phase
FOR each uncommitted transaction T DO

    Read its log records in reverse order

    FOR each log record of T
        UNDO the operation
    END FOR

END FOR

Write recovery information to the log

PRINT "Database Recovery Completed"

END
```

Example

Consider the log:

<T1 START>

<T1, A, 100, 150>

<T1 COMMIT>

<T2 START>

<T2, B, 200, 250>

SYSTEM CRASH

At the time of failure:

T1 → Committed

T2 → Uncommitted

Recovery

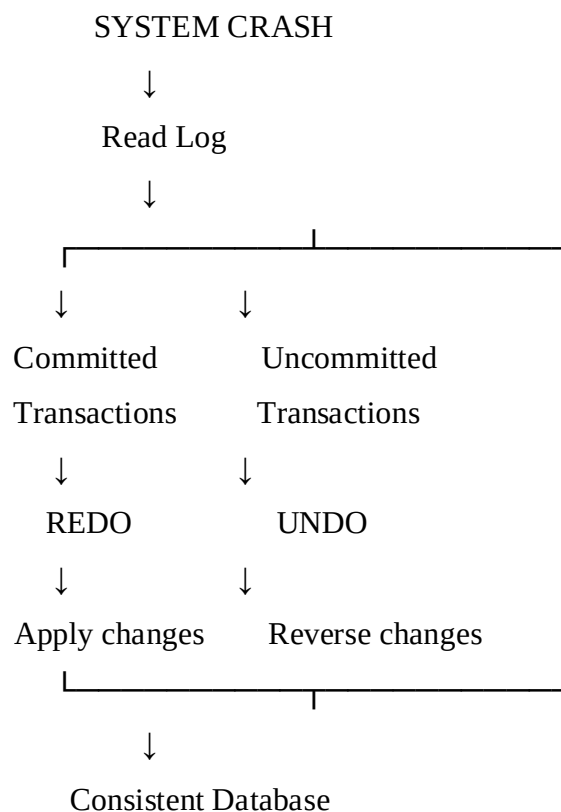
REDO T1:

A = 150

UNDO T2:

B = 250 → 200

Recovery Process



Result

In **Immediate Update recovery**, committed transactions are **REDONE** and uncommitted transactions are **UNDONE**. This restores the database to a consistent state after a system crash.

Q10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

This question asks for a **database connectivity manager** that handles three main tasks: **connection pooling, query execution, and exception rollback.**

Pseudocode

ALGORITHM Database_Connectivity_Manager

INPUT:

SQL Query

Query Parameters

OUTPUT:

Query Result

BEGIN

Initialize Connection Pool

TRY

// Get connection from pool

Connection ← GetConnection()

Begin Transaction

// Execute Query

Prepare SQL Query

Set Query Parameters

Result ← Execute Query

IF Query executed successfully THEN

COMMIT Transaction

Return Connection to Pool

RETURN Result

ELSE

ROLLBACK Transaction

Return Connection to Pool

RETURN Error

END IF

CATCH Exception

ROLLBACK Transaction

Return Connection to Pool

PRINT "Database Error"

PRINT Exception Message

END TRY

END

Connection Pooling

Initialize Pool



Create Connections



Store Connections in Pool



Get Available Connection



Execute Query



Return Connection to Pool

Instead of creating a new database connection for every query, an existing connection from the pool is reused.

Query Execution

Get Connection



Prepare SQL Query



Set Parameters



Execute Query



Get Result

Exception Rollback

If an error occurs during execution:

Query Error



ROLLBACK



Cancel Changes



Return Connection



Report Error

Example

T1: BEGIN TRANSACTION

Execute:

UPDATE Employee

SET Salary = Salary + 5000

WHERE Eid = 101

If successful:

COMMIT

If exception occurs:

ROLLBACK

Result

The database connectivity manager efficiently manages database connections using **connection pooling**, executes SQL queries using an available connection, and performs **rollback when an exception occurs**. This improves resource utilization and maintains database consistency.